

Computação Paralela e Distribuída 2025/2026

- Procura do maior número primo em tempo limitado,
simulação do Game of Life e exposição de funcionalidades
via RPC com sockets -

Trabalho Prático

Turma:	Grupo:	Número:	Nome:
PL2	G04	2024151350	Diogo Brito
PL2	G04	2024148451	Diogo Gomes
PL2	G04	2024151531	Rafael Junqueira

Data: 06/2026

Índice

1.	Descrição da Solução.....	3
2.	Paralelização e Estratégias de Divisão	4
3.	Sistema Distribuído (RPC)	5
4.	Análise de Desempenho	6
5.	Conclusão	7

1. Descrição da Solução

O objetivo deste trabalho foi desenvolver um sistema capaz de realizar cálculos intensivos de forma paralela e expor essas funcionalidades através de uma rede utilizando um sistema de RPC (*Remote Procedure Call*) customizado.

A solução foi organizada de forma modular:

- **primos.py**: Contém a lógica de procura de números primos.
- **game_of_life.py**: Implementa o autômato celular de Conway.
- **servidor.py**: Gere as ligações de rede e invoca as funções.
- **cliente.py**: Fornece uma interface de utilizador para pedidos remotos.
- **testes.py**: Script de validação automática que garante o correto funcionamento das funções e a consistência entre as versões sequenciais e paralelas.

2. Paralelização e Estratégias de Divisão

2.1 Escolha do Modelo: Threads vs Processos

Para as tarefas de cálculo (primos e `game_of_life`), optou-se pela utilização do módulo **multiprocessing**. Esta decisão baseia-se na existência do **GIL (Global Interpreter Lock)** no CPython, que impede a execução simultânea de múltiplas threads em tarefas *CPU-bound*. A utilização de processos independentes permite contornar esta limitação e utilizar todos os núcleos do CPU.

2.2 Procura de Números Primos

- **Estratégia:** Divisão do espaço de procura por "saltos" (*stride*). Se tivermos 4 workers, o Worker 1 testa os números 3, 11, 15... o Worker 2 testa 5, 13, 17, e assim sucessivamente.
- **Sincronização:** Utilizou-se um objeto `multiprocessing.Value` protegido por um `multiprocessing.Lock` para garantir que a atualização do maior primo encontrado seja atômica, evitando *race conditions*.

2.3 Game of Life

- **Estratégia:** Divisão da grelha em regiões horizontais (fatias de linhas). Cada processo é responsável por calcular o próximo estado de um conjunto de linhas.
- **Sincronização:** Utilizou-se um `Pool.starmap` que garante uma barreira de sincronização natural entre gerações: nenhuma linha da geração $N + 1$ começa a ser calculada até que todas as linhas da geração N estejam prontas.

3. Sistema Distribuído (RPC)

3.1 Arquitetura e Comunicação

O sistema segue o modelo **Cliente-Servidor** sobre sockets **TCP**.

- **Concorrência:** O servidor utiliza o módulo `threading` para gerir as conexões. Como a rede é uma tarefa *I/O-bound*, as threads são eficientes para atender múltiplos clientes em simultâneo enquanto os processos tratam dos cálculos.
- **Protocolo:** Implementou-se um protocolo baseado em **JSON-RPC 2.0**. As mensagens são serializadas em JSON, codificadas em UTF-8 e enviadas como sequências de bytes.

3.2 Introspeção e Dispatch Dinâmico

O servidor utiliza o módulo `inspect` para identificar automaticamente as funções disponíveis em `FUNCOES_REMOTAS`. Isto permite que o método `list_methods` devolva ao cliente a assinatura e a descrição (*docstring*) de cada função sem necessidade de configuração manual.

4. Análise de Desempenho

Procura de Números Primos (Timeout: 2s)

```
Procura iniciada (2s)...  
Sequencial: 1461311  
Paralelo: 3007937
```

Métrica	Versão Sequencial	Versão Paralela (4 workers)
Maior Primo Encontrado	1.461.311	3.007.937
N.º de Algarismos	7 algarismos	7 algarismos
Ganho de Exploração	-	+105.8%

Como o problema é limitado pelo tempo e não por um número fixo, a métrica de sucesso é a profundidade da procura. A versão paralela conseguiu ultrapassar a barreira dos 3 milhões, enquanto a sequencial ficou abaixo de 1.5 milhões. Este ganho de aproximadamente **2.05x** demonstra que o uso de múltiplos processos permitiu distribuir o teste de candidatos a primo pelos vários núcleos do CPU de forma eficiente. O ganho não é perfeitamente linear (4x) devido ao custo de comunicação e sincronização do Lock ao atualizar o valor partilhado.

5. Conclusão

A implementação demonstrou que o paralelismo com processos é vital para o ganho de performance em Python em tarefas de cálculo. O sistema de RPC desenvolvido provou ser robusto, suportando erros de utilizador e garantindo a consistência dos dados através da rede. O *overhead* de criação de processos foi minimizado pelo uso de Pools, tornando a solução escalável para problemas de maior dimensão.